In [131…

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
import sklearn.metrics as metrics

from sklearn import tree
from sklearn.tree import _tree

from sklearn.ensemble import RandomForestRegressor
from sklearn.ensemble import RandomForestClassifier

from sklearn.ensemble import GradientBoostingRegressor
from sklearn.ensemble import GradientBoostingClassifier

import math
from operator import itemgetter

sns.set()

pd.set_option('display.max_rows', None)
pd.set_option('display.max_columns', None)
pd.set_option('display.width', None)
pd.set_option('display.max_colwidth', None)
```

Load In Dataset

In [132…

```
#read in dataset file and create dataframe

file = "C:\\Users\\Ivan\\Documents\\MSDS422\\Assignment 1\\HMEQ_Loss.csv"

df = pd.read_csv( file )
```

Organize Fields, Replace Missing Values, and One Hot Encoding

In [133…

```
#organize fields into lists by type
#leaving out the two target variables

dt = df.dtypes

TARGET_F = "TARGET_BAD_FLAG"
TARGET_A = "TARGET_LOSS_AMT"

objList = []
intList = []
floatList = []
```

```
for i in dt.index:
    if i in ([TARGET_F, TARGET_A]):continue
    if dt[i] in (["object"]):objList.append(i)
    if dt[i] in (["float64"]):floatList.append(i)
    if dt[i] in (["int64"]):intList.append(i)
```

In [134…
```
for i in objList:
    if df[i].isna().sum() == 0:continue
    Name = "IMP_" + i
    df[Name] = df[i] #gives the copied variable the original variable's data
    df[Name] = df[Name].fillna("Missing")
    g = df.groupby(Name)
```

In [135…
```
for i in intList:
   if df[i].isna().sum() == 0:continue
   Flag = "M_"+i
   Imp = "IMP_"+i
   df[Flag] = df[i].isna() + 0
   df[Imp] = df[i]
   df.loc[df[Imp].isna(), Imp] = df[i].median()
```

In [136…
```
for i in floatList:
   if df[i].isna().sum() == 0:continue
   Flag = "M_"+i
   Imp = "IMP_"+i
   df[Flag] = df[i].isna() + 0
   df[Imp] = df[i]
   df.loc[df[Imp].isna(), Imp] = df[i].median()
```

In [137…
```
for i in ["IMP_REASON", "IMP_JOB"]:
    prefix = "z_"+i
    y = pd.get_dummies(df[i], prefix=prefix, dummy_na = 0, dtype=int)
    print(y.head().T)
    df = pd.concat([df,y], axis = 1)
    df = df.loc[:, ~df.columns.duplicated()]
```

```
                        0  1  2  3  4
z_IMP_REASON_DebtCon    0  0  0  0  0
z_IMP_REASON_HomeImp    1  1  1  0  1
z_IMP_REASON_Missing    0  0  0  1  0
                        0  1  2  3  4
z_IMP_JOB_Mgr           0  0  0  0  0
z_IMP_JOB_Missing       0  0  0  1  0
z_IMP_JOB_Office        0  0  0  0  1
z_IMP_JOB_Other         1  1  1  0  0
z_IMP_JOB_ProfExe       0  0  0  0  0
z_IMP_JOB_Sales         0  0  0  0  0
z_IMP_JOB_Self          0  0  0  0  0
```

In [138…
```
df_clean = df[sorted(df.columns)]

column = df_clean.pop(TARGET_F)
df_clean.insert(0,TARGET_F, column)
```

```
column = df_clean.pop(TARGET_A)
df_clean.insert(1,TARGET_A, column)

df_clean = df_clean.drop(columns = ["JOB", "REASON", "CLAGE","CLNO","DEBTINC","DELI
                                    "DEROG","MORTDUE","NINQ","VALUE","YOJ"], axis = 1)

print(df_clean.head().T)
```

	0	1	2	3	4
TARGET_BAD_FLAG	1	1	1	1	0
TARGET_LOSS_AMT	641.0	1109.0	767.0	1425.0	NaN
IMP_CLAGE	94.366667	121.833333	149.466667	173.466667	93.333333
IMP_CLNO	9.0	14.0	10.0	20.0	14.0
IMP_DEBTINC	34.818262	34.818262	34.818262	34.818262	34.818262
IMP_DELINQ	0.0	2.0	0.0	0.0	0.0
IMP_DEROG	0.0	0.0	0.0	0.0	0.0
IMP_JOB	Other	Other	Other	Missing	Office
IMP_MORTDUE	25860.0	70053.0	13500.0	65019.0	97800.0
IMP_NINQ	1.0	0.0	1.0	1.0	0.0
IMP_REASON	HomeImp	HomeImp	HomeImp	Missing	HomeImp
IMP_VALUE	39025.0	68400.0	16700.0	89235.5	112000.0
IMP_YOJ	10.5	7.0	4.0	7.0	3.0
LOAN	1100	1300	1500	1500	1700
M_CLAGE	0	0	0	1	0
M_CLNO	0	0	0	1	0
M_DEBTINC	1	1	1	1	1
M_DELINQ	0	0	0	1	0
M_DEROG	0	0	0	1	0
M_MORTDUE	0	0	0	1	0
M_NINQ	0	0	0	1	0
M_VALUE	0	0	0	1	0
M_YOJ	0	0	0	1	0
z_IMP_JOB_Mgr	0	0	0	0	0
z_IMP_JOB_Missing	0	0	0	1	0
z_IMP_JOB_Office	0	0	0	0	1
z_IMP_JOB_Other	1	1	1	0	0
z_IMP_JOB_ProfExe	0	0	0	0	0
z_IMP_JOB_Sales	0	0	0	0	0
z_IMP_JOB_Self	0	0	0	0	0
z_IMP_REASON_DebtCon	0	0	0	0	0
z_IMP_REASON_HomeImp	1	1	1	0	1
z_IMP_REASON_Missing	0	0	0	1	0

In [139…

```
print(df_clean.describe().T)
```

	count	mean	std	min	\
TARGET_BAD_FLAG	5960.0	0.199497	0.399656	0.000000	
TARGET_LOSS_AMT	1189.0	13414.576955	10839.455965	224.000000	
IMP_CLAGE	5960.0	179.440725	83.574697	0.000000	
IMP_CLNO	5960.0	21.247819	9.951308	0.000000	
IMP_DEBTINC	5960.0	34.000651	7.644528	0.524499	
IMP_DELINQ	5960.0	0.405705	1.079256	0.000000	
IMP_DEROG	5960.0	0.224329	0.798458	0.000000	
IMP_MORTDUE	5960.0	73001.041812	42552.726779	2063.000000	
IMP_NINQ	5960.0	1.170134	1.653866	0.000000	
IMP_VALUE	5960.0	101540.387423	56869.436682	8000.000000	
IMP_YOJ	5960.0	8.756166	7.259424	0.000000	
LOAN	5960.0	18607.969799	11207.480417	1100.000000	
M_CLAGE	5960.0	0.051678	0.221394	0.000000	
M_CLNO	5960.0	0.037248	0.189386	0.000000	
M_DEBTINC	5960.0	0.212584	0.409170	0.000000	
M_DELINQ	5960.0	0.097315	0.296412	0.000000	
M_DEROG	5960.0	0.118792	0.323571	0.000000	
M_MORTDUE	5960.0	0.086913	0.281731	0.000000	
M_NINQ	5960.0	0.085570	0.279752	0.000000	
M_VALUE	5960.0	0.018792	0.135801	0.000000	
M_YOJ	5960.0	0.086409	0.280991	0.000000	
z_IMP_JOB_Mgr	5960.0	0.128691	0.334886	0.000000	
z_IMP_JOB_Missing	5960.0	0.046812	0.211254	0.000000	
z_IMP_JOB_Office	5960.0	0.159060	0.365763	0.000000	
z_IMP_JOB_Other	5960.0	0.400671	0.490076	0.000000	
z_IMP_JOB_ProfExe	5960.0	0.214094	0.410227	0.000000	
z_IMP_JOB_Sales	5960.0	0.018289	0.134004	0.000000	
z_IMP_JOB_Self	5960.0	0.032383	0.177029	0.000000	
z_IMP_REASON_DebtCon	5960.0	0.659060	0.474065	0.000000	
z_IMP_REASON_HomeImp	5960.0	0.298658	0.457708	0.000000	
z_IMP_REASON_Missing	5960.0	0.042282	0.201248	0.000000	

	25%	50%	75%	max
TARGET_BAD_FLAG	0.000000	0.000000	0.000000	1.000000
TARGET_LOSS_AMT	5639.000000	11003.000000	17634.000000	78987.000000
IMP_CLAGE	117.371430	173.466667	227.143058	1168.233561
IMP_CLNO	15.000000	20.000000	26.000000	71.000000
IMP_DEBTINC	30.763159	34.818262	37.949892	203.312149
IMP_DELINQ	0.000000	0.000000	0.000000	15.000000
IMP_DEROG	0.000000	0.000000	0.000000	10.000000
IMP_MORTDUE	48139.000000	65019.000000	88200.250000	399550.000000
IMP_NINQ	0.000000	1.000000	2.000000	17.000000
IMP_VALUE	66489.500000	89235.500000	119004.750000	855909.000000
IMP_YOJ	3.000000	7.000000	12.000000	41.000000
LOAN	11100.000000	16300.000000	23300.000000	89900.000000
M_CLAGE	0.000000	0.000000	0.000000	1.000000
M_CLNO	0.000000	0.000000	0.000000	1.000000
M_DEBTINC	0.000000	0.000000	0.000000	1.000000
M_DELINQ	0.000000	0.000000	0.000000	1.000000
M_DEROG	0.000000	0.000000	0.000000	1.000000
M_MORTDUE	0.000000	0.000000	0.000000	1.000000
M_NINQ	0.000000	0.000000	0.000000	1.000000
M_VALUE	0.000000	0.000000	0.000000	1.000000
M_YOJ	0.000000	0.000000	0.000000	1.000000
z_IMP_JOB_Mgr	0.000000	0.000000	0.000000	1.000000

```
z_IMP_JOB_Missing        0.000000      0.000000       0.000000       1.000000
z_IMP_JOB_Office         0.000000      0.000000       0.000000       1.000000
z_IMP_JOB_Other          0.000000      0.000000       1.000000       1.000000
z_IMP_JOB_ProfExe        0.000000      0.000000       0.000000       1.000000
z_IMP_JOB_Sales          0.000000      0.000000       0.000000       1.000000
z_IMP_JOB_Self           0.000000      0.000000       0.000000       1.000000
z_IMP_REASON_DebtCon     0.000000      1.000000       1.000000       1.000000
z_IMP_REASON_HomeImp     0.000000      0.000000       1.000000       1.000000
z_IMP_REASON_Missing     0.000000      0.000000       0.000000       1.000000
```

In [140…
```
df_clean.select_dtypes(include='object').columns
```

Out[140…
```
Index(['IMP_JOB', 'IMP_REASON'], dtype='object')
```

Create Training and Test Data Set

In [141…
```
X = df_clean.copy()
X = pd.get_dummies(X, drop_first=True) #one hot encoding to make sure decision tree

X = X.drop( TARGET_F, axis = 1)
X = X.drop( TARGET_A, axis = 1) # X is the dataset without the targets for the mode

Y = df_clean[[TARGET_F, TARGET_A]]# Y is the dataset with only the targets for the

#we use X to predict Y

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, train_size=0.8, test_size
```

In [142…
```
print("FLAG DATA")
print("TRAINING = ", X_train.shape)
print("TEST = ", X_test.shape) # check if the (rows, fields) make sense

# do the number of rows between training and testing look like an 80-20 split?
# are the number of fields the same between the training and testing data?
```

```
FLAG DATA
TRAINING =  (4768, 37)
TEST =  (1192, 37)
```

Decision Trees

In [143…
```
fm01_Tree = tree.DecisionTreeClassifier(max_depth = 5)
fm01_Tree = fm01_Tree.fit(X_train, Y_train[TARGET_F])
```

In [144…
```
Y_Pred_train = fm01_Tree.predict(X_train)
Y_Pred_test = fm01_Tree.predict(X_test)
```

In [145…
```
print("Accuracy of Training Dataset:", metrics.accuracy_score(Y_train[TARGET_F], Y_
print("Accuracy of Testing Dataset:", metrics.accuracy_score(Y_test[TARGET_F], Y_Pr
```

```
Accuracy of Training Dataset: 0.8974412751677853
Accuracy of Testing Dataset: 0.886744966442953
```

In [146…
```
feature_cols = list(X.columns.values)
tree.export_graphviz(fm01_Tree, out_file='Assignment_2_MSDS422_decision_tree.txt',
                     filled = True, rounded = True,
                     feature_names = feature_cols, impurity = False,
                     class_names=["Good","Bad"])
```

In [147…
```
def getTreeVars(TREE, varNames):
    tree_ = TREE.tree_
    varName = [varNames[i] if i != _tree.TREE_UNDEFINED else "undefined!" for i in
    nameSet = set()
    for i in tree_.feature:
        if i != _tree.TREE_UNDEFINED:
            nameSet.add(i)
    nameList = list(nameSet)
    parameter_list = list()
    for i in nameList:
        parameter_list.append(varNames[i])
    return parameter_list
```

In [148…
```
vars_tree_flag = getTreeVars(fm01_Tree, feature_cols)
```

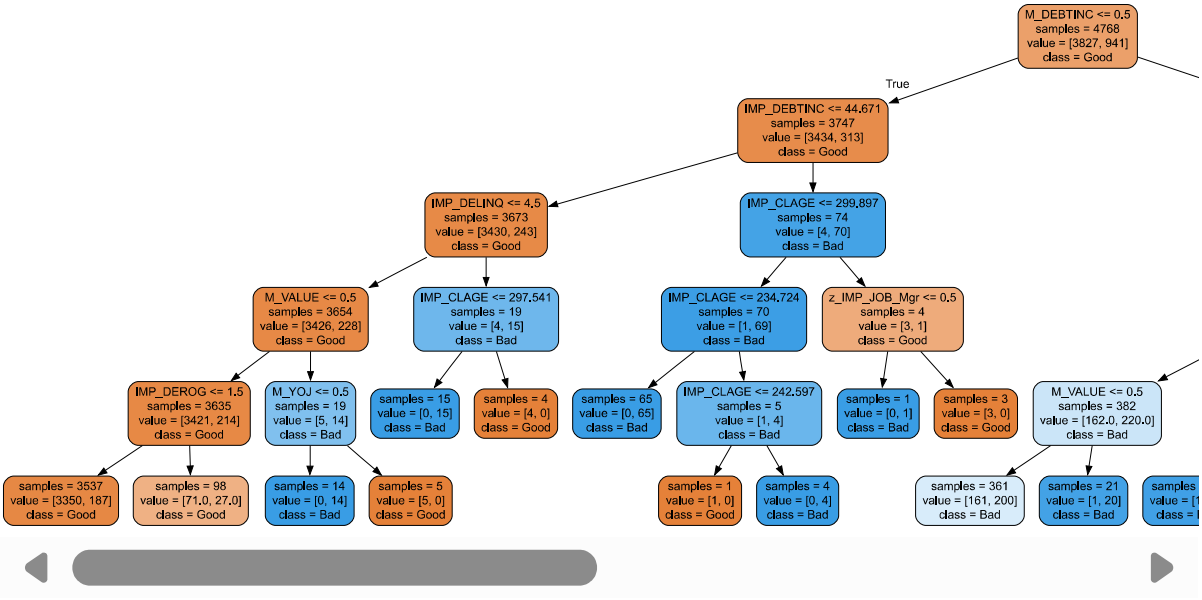
In [149…
```
for i in vars_tree_flag: #seeing which variables the tree used to predict loan defa
    print(i)
```

```
IMP_CLAGE
IMP_DEBTINC
IMP_DELINQ
IMP_DEROG
IMP_VALUE
IMP_YOJ
M_DEBTINC
M_DEROG
M_VALUE
M_YOJ
z_IMP_JOB_Mgr
z_IMP_JOB_Sales
```

In [150…
```
from graphviz import Source

with open("Assignment_2_MSDS422_decision_tree.txt") as f:
    dot_data = f.read()

dot_data = dot_data.replace(
    "digraph Tree {",
    'digraph Tree {\n  size="6,8";\n  dpi=150;'
)

Source(dot_data)
```

```
Warning: Could not load "C:\Users\Ivan\anaconda3\Destination\Library\bin\gvplugin_pa
ngo.dll" - It was found, so perhaps one of its dependents was not.  Try ldd.
```

Out[150…



Regression Tree

In [151…
```
F = ~ Y_train[TARGET_A].isna() #get rid of missing values in Target A for regressio
W_train = X_train[F]
Z_train = Y_train[F]
```

In [152…
```
F = ~ Y_test[TARGET_A].isna() #get rid of missing values in Target A for regression
W_test = X_test[F]
Z_test = Y_test[F]
```

In [153…
```
amt_m01_Tree = tree.DecisionTreeRegressor(max_depth=5)
amt_m01_Tree = amt_m01_Tree.fit(W_train, Z_train[TARGET_A])
```

In [154…
```
Z_Pred_train = amt_m01_Tree.predict(W_train)
Z_Pred_test = amt_m01_Tree.predict(W_test)
```

In [155…
```
RMSE_train = math.sqrt(metrics.mean_squared_error(Z_train[TARGET_A], Z_Pred_train))
RMSE_test = math.sqrt(metrics.mean_squared_error(Z_test[TARGET_A], Z_Pred_test))

print("TREE RMSE Train:", RMSE_train)
print("TREE RMSE Test:", RMSE_test)
```

```
TREE RMSE Train: 3669.612222739016
TREE RMSE Test: 5410.350018411686
```

In [156…
```
RMSE_tree = RMSE_test
```

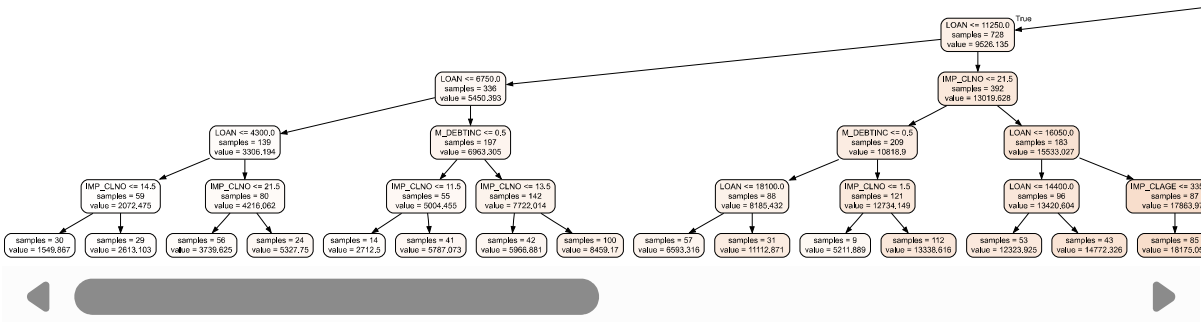
In [157…
```
tree.export_graphviz(amt_m01_Tree, out_file='Assignment_2_MSDS422_regression_tree.t
                     filled = True, rounded = True,
                     feature_names = feature_cols, impurity = False,
                     class_names=["Good","Bad"])
```

In [158…
```
from graphviz import Source

with open("Assignment_2_MSDS422_regression_tree.txt") as f:
```

```
    dot_data = f.read()

dot_data = dot_data.replace(
    "digraph Tree {",
    'digraph Tree {\n  size="6,8";\n  dpi=150;'
)

Source(dot_data)
```

```
Warning: Could not load "C:\Users\Ivan\anaconda3\Destination\Library\bin\gvplugin_pa
ngo.dll" - It was found, so perhaps one of its dependents was not.  Try ldd.
```

Out[158…



In [159…
```
vars_tree_amt = getTreeVars(amt_m01_Tree, feature_cols)
```

In [160…
```
for i in vars_tree_amt:
    print(i)
```

```
IMP_CLAGE
IMP_CLNO
IMP_DEBTINC
IMP_DELINQ
IMP_MORTDUE
LOAN
M_DEBTINC
z_IMP_REASON_DebtCon
IMP_JOB_Office
IMP_JOB_Other
```

Random Forests

In [161…
```
fm01_RF = RandomForestClassifier(n_estimators = 100, random_state = 67)
fm01_RF = fm01_RF.fit(X_train, Y_train[TARGET_F])
```

In [162…
```
Y_pred_train_RF = fm01_RF.predict(X_train)
Y_pred_test_RF = fm01_RF.predict(X_test)
```

In [163…
```
print("Accuracy of Training Dataset:", metrics.accuracy_score(Y_train[TARGET_F], Y_
print("Accuracy of Testing Dataset:", metrics.accuracy_score(Y_test[TARGET_F], Y_pr
```

```
Accuracy of Training Dataset: 1.0
Accuracy of Testing Dataset: 0.9194630872483222
```

In [164…
```
probs = fm01_RF.predict_proba(X_train)
p1_RF = probs[:,1]
```

```
fpr_train_RF, tpr_train_RF, threshold = metrics.roc_curve(Y_train[TARGET_F], p1_RF)
roc_auc_train_RF = metrics.auc(fpr_train_RF, tpr_train_RF)
```

In [165…
```
probs = fm01_RF.predict_proba(X_test)
p1_RF_test = probs[:,1]
fpr_test_RF, tpr_test_RF, threshold = metrics.roc_curve(Y_test[TARGET_F], p1_RF_tes
roc_auc_test_RF = metrics.auc(fpr_test_RF, tpr_test_RF)
```

In [166…
```
fpr_RF = fpr_test_RF
tpr_RF = tpr_test_RF
auc_RF = roc_auc_test_RF
```

In [167…
```
def getEnsembleTreeVars(ENSTREE, varNames):
    importance = ENSTREE.feature_importances_
    index = np.argsort(importance)
    theList = []
    for i in index:
        imp_val = importance[i]
        if imp_val > np.average(ENSTREE.feature_importances_):
            v = int(imp_val/np.max(ENSTREE.feature_importances_) * 100)
            theList.append((varNames[i],v))
    theList = sorted(theList, key=itemgetter(1), reverse = True)
    return theList
```

In [168…
```
vars_RF_flag = getEnsembleTreeVars(fm01_RF, feature_cols)
```

In [169…
```
for i in vars_RF_flag:
    print(i)
```

```
('M_DEBTINC', 100)
('IMP_DEBTINC', 80)
('IMP_CLAGE', 47)
('IMP_DELINQ', 45)
('LOAN', 41)
('IMP_VALUE', 39)
('IMP_MORTDUE', 36)
('IMP_CLNO', 36)
('IMP_YOJ', 30)
('IMP_DEROG', 25)
('IMP_NINQ', 21)
```

RMSE Random Forest

In [170…
```
amt_m01_RF = RandomForestRegressor(n_estimators = 100, random_state=1)
amt_m01_RF = amt_m01_RF.fit(W_train, Z_train[TARGET_A])
```

In [171…
```
Z_pred_train_RF = amt_m01_RF.predict(W_train)
Z_pred_test_RF = amt_m01_RF.predict(W_test)
```

In [172…
```
RMSE_train_RF = math.sqrt(metrics.mean_squared_error(Z_train[TARGET_A], Z_pred_trai
RMSE_test_RF = math.sqrt(metrics.mean_squared_error(Z_test[TARGET_A], Z_pred_test_R
```

In [173…
```
print("RF RMSE Train:", RMSE_train_RF)
print("RF RMSE Test:", RMSE_test_RF)
```

```
RF RMSE Train: 1232.837683643201
RF RMSE Test: 3225.6898950862105
```

In [174…
```
RMSE_RF = RMSE_test_RF
```

In [175…
```
vars_RF_amt = getEnsembleTreeVars(amt_m01_RF, feature_cols)
```

In [176…
```
for i in vars_RF_amt:
    print(i)
```

```
('LOAN', 100)
('IMP_CLNO', 12)
('IMP_DEBTINC', 5)
('M_DEBTINC', 4)
```

Gradient Boosting Model

In [177…
```
fm01_GB = GradientBoostingClassifier(random_state = 1)
fm01_GB = fm01_GB.fit(X_train, Y_train[TARGET_F])
```

In [178…
```
Y_pred_train_GB = fm01_GB.predict(X_train)
Y_pred_test_GB = fm01_GB.predict(X_test)
```

In [179…
```
print("Accuracy of Training Dataset:", metrics.accuracy_score(Y_train[TARGET_F], Y_
print("Accuracy of Testing Dataset:", metrics.accuracy_score(Y_test[TARGET_F], Y_pr
```

```
Accuracy of Training Dataset: 0.9234479865771812
Accuracy of Testing Dataset: 0.9043624161073825
```

In [180…
```
probs = fm01_GB.predict_proba(X_train)
p1_gb = probs[:,1]
fpr_train_GB, tpr_train_GB, threshold = metrics.roc_curve(Y_train[TARGET_F], p1_gb)
roc_auc_train_GB = metrics.auc(fpr_train_GB, tpr_train_GB)
```

In [181…
```
probs = fm01_GB.predict_proba(X_test)
p1_gb = probs[:,1]
fpr_test_GB, tpr_test_GB, threshold = metrics.roc_curve(Y_test[TARGET_F], p1_gb)
roc_auc_test_GB = metrics.auc(fpr_test_GB, tpr_test_GB)
```

In [182…
```
fpr_GB = fpr_test_GB
tpr_GB = tpr_test_GB
auc_GB = roc_auc_test_GB
```

In [183…
```
vars_GB_flag = getEnsembleTreeVars(fm01_GB, feature_cols)
```

In [184…
```
for i in vars_GB_flag:
    print(i)
```

```
('M_DEBTINC', 100)
('IMP_DEBTINC', 29)
('IMP_DELINQ', 19)
('IMP_CLAGE', 14)
('M_VALUE', 7)
('IMP_DEROG', 7)
```

RMSE Gradient Boosting Model

In [185…
```
amt_m01_GB = GradientBoostingRegressor(random_state = 1)
amt_m01_GB = amt_m01_GB.fit(W_train, Z_train[TARGET_A])
```

In [186…
```
Z_pred_train_GB = amt_m01_GB.predict(W_train)
Z_pred_test_GB = amt_m01_GB.predict(W_test)
```

In [187…
```
RMSE_train_GB = math.sqrt(metrics.mean_squared_error(Z_train[TARGET_A], Z_pred_trai
RMSE_test_GB = math.sqrt(metrics.mean_squared_error(Z_test[TARGET_A], Z_pred_test_G
```

In [188…
```
print("GB RMSE Train:", RMSE_train_GB)
print("GB RMSE Test:", RMSE_test_GB)
```

```
GB RMSE Train: 1190.372223849314
GB RMSE Test: 2614.670782701094
```

In [189…
```
RMSE_GB = RMSE_test_GB
```

In [190…
```
vars_GB_amt = getEnsembleTreeVars(amt_m01_GB, feature_cols)
```

In [191…
```
for i in vars_GB_amt:
    print(i)
```

```
('LOAN', 100)
('IMP_CLNO', 14)
('IMP_DEBTINC', 5)
('M_DEBTINC', 5)
```

ROC Curves

Decision Tree ROC Curve

In [192…
```
probs = fm01_Tree.predict_proba(X_train) #matrix with the probability of a loan not

p1 = probs[:,1] # creating a dataset with just the probabilities of a loan defaulti

p1[0:5] # the probability a loan will default for the first 5 loans in the dataset
```

Out[192…
```
array([0.94957983, 0.80803571, 0.05286966, 0.55401662, 0.05286966])
```

In [193…
```
fpr_train, tpr_train, threshold = metrics.roc_curve(Y_train[TARGET_F], p1)
#fpr = false positive rate
#tpr = true positive rate
```

In [194…
```
roc_auc_train = metrics.auc(fpr_train, tpr_train)
```

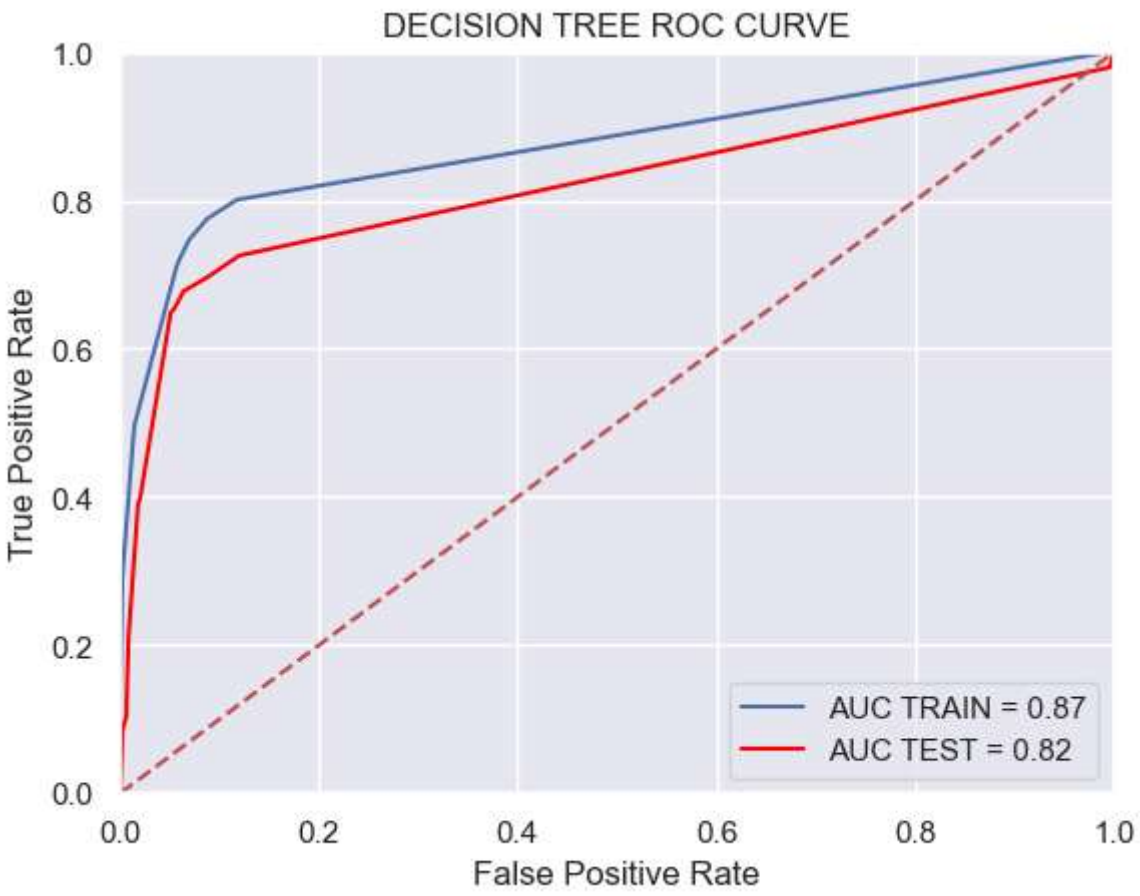
In [195…
```
probs = fm01_Tree.predict_proba(X_test) #matrix with the probability of a loan not

p2 = probs[:,1] # creating a dataset with just the probabilities of a loan defaulti
```

In [196…
```
fpr_test, tpr_test, threshold = metrics.roc_curve(Y_test[TARGET_F], p2)
```

In [197…
```
roc_auc_test = metrics.auc(fpr_test, tpr_test)
```

In [198…
```
fpr_tree = fpr_test
tpr_tree = tpr_test
auc_tree = roc_auc_test
```

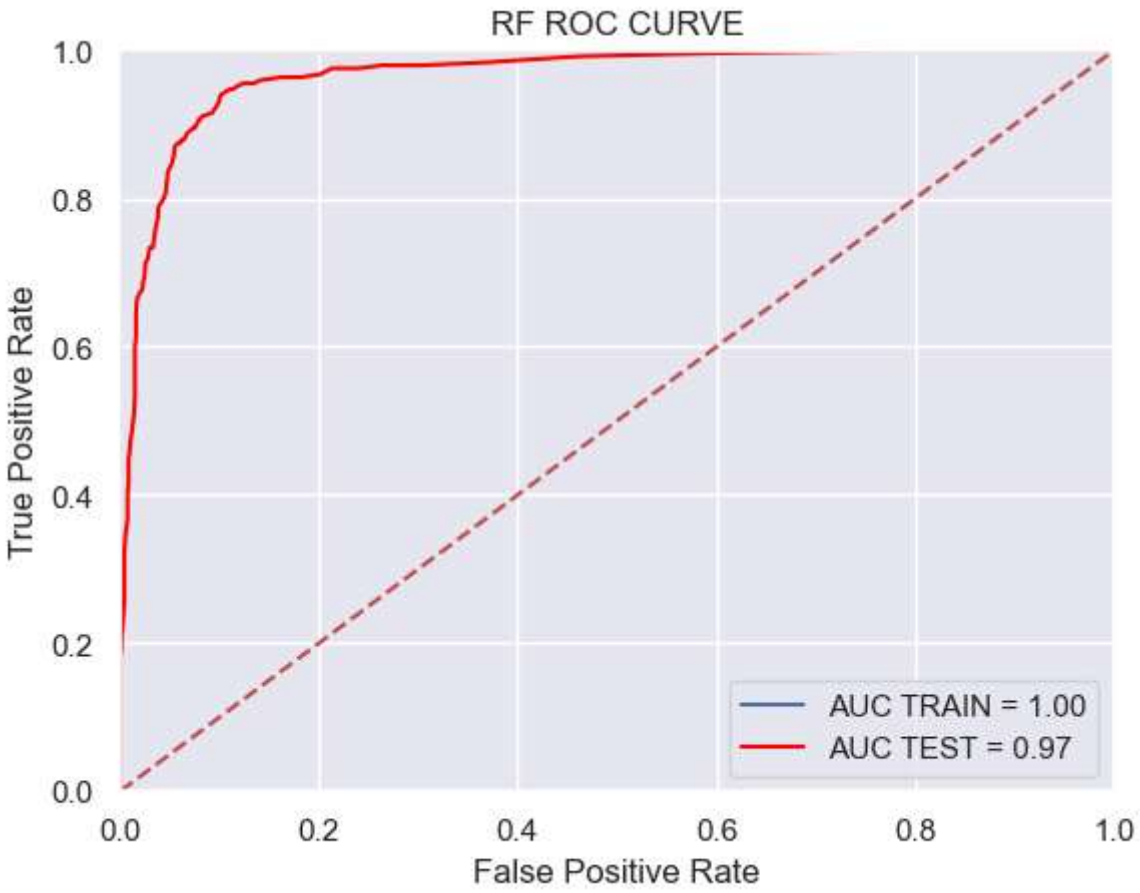
In [199…
```
plt.title("DECISION TREE ROC CURVE")
plt.plot(fpr_train, tpr_train, "b", label = "AUC TRAIN = %0.2f" % roc_auc_train)
plt.plot(fpr_test, tpr_test, label = "AUC TEST = %0.2f" % roc_auc_test, color = "re
plt.legend(loc = "lower right")
plt.plot([0,1], [0,1], "r--")
plt.xlim([0,1])
plt.ylim([0,1])
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.show()
```



Random Forest ROC curve

In [200…

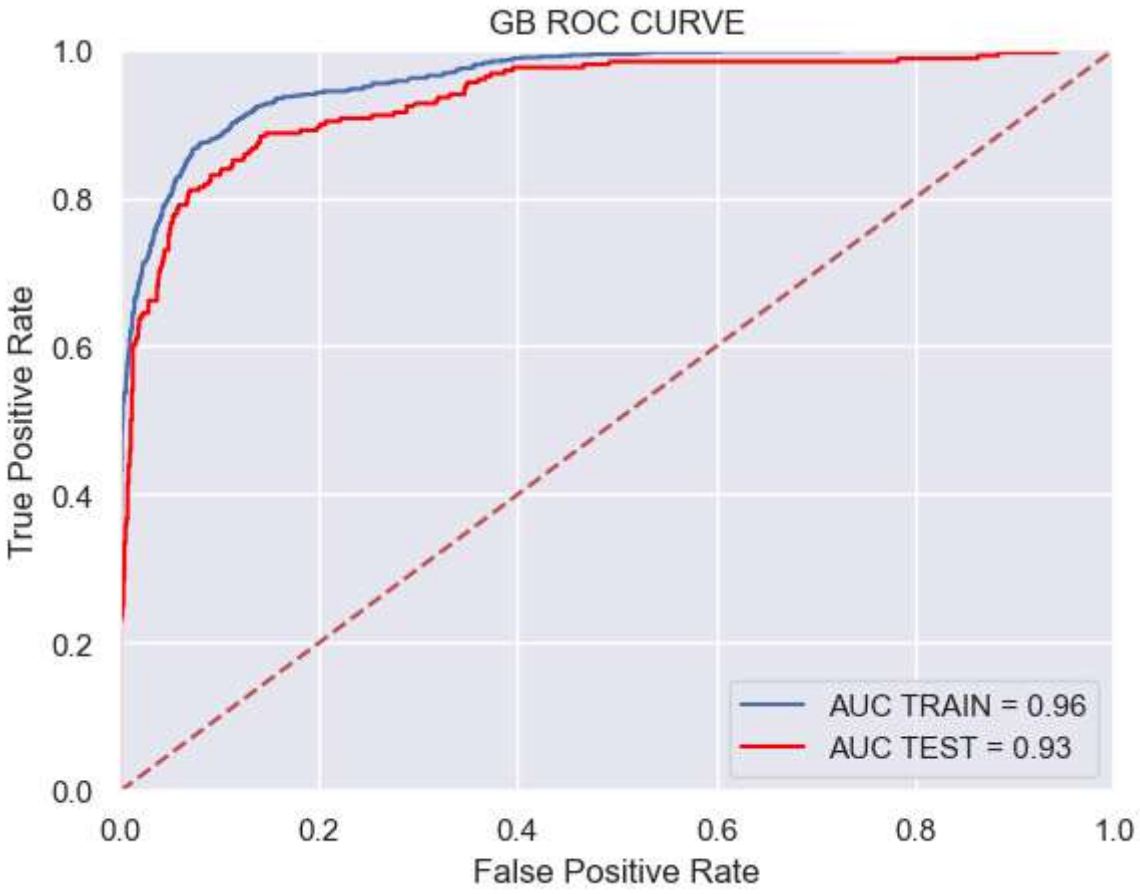
```
plt.title('RF ROC CURVE')
plt.plot(fpr_train_RF, tpr_train_RF, "b", label = "AUC TRAIN = %0.2f" % roc_auc_tra
plt.plot(fpr_test_RF, tpr_test_RF, label = "AUC TEST = %0.2f" % roc_auc_test_RF, co
plt.legend(loc = "lower right")
plt.plot([0,1], [0,1], "r--")
plt.xlim([0,1])
plt.ylim([0,1])
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.show()
```



Gradient Boosting Model ROC Curve

In [201…

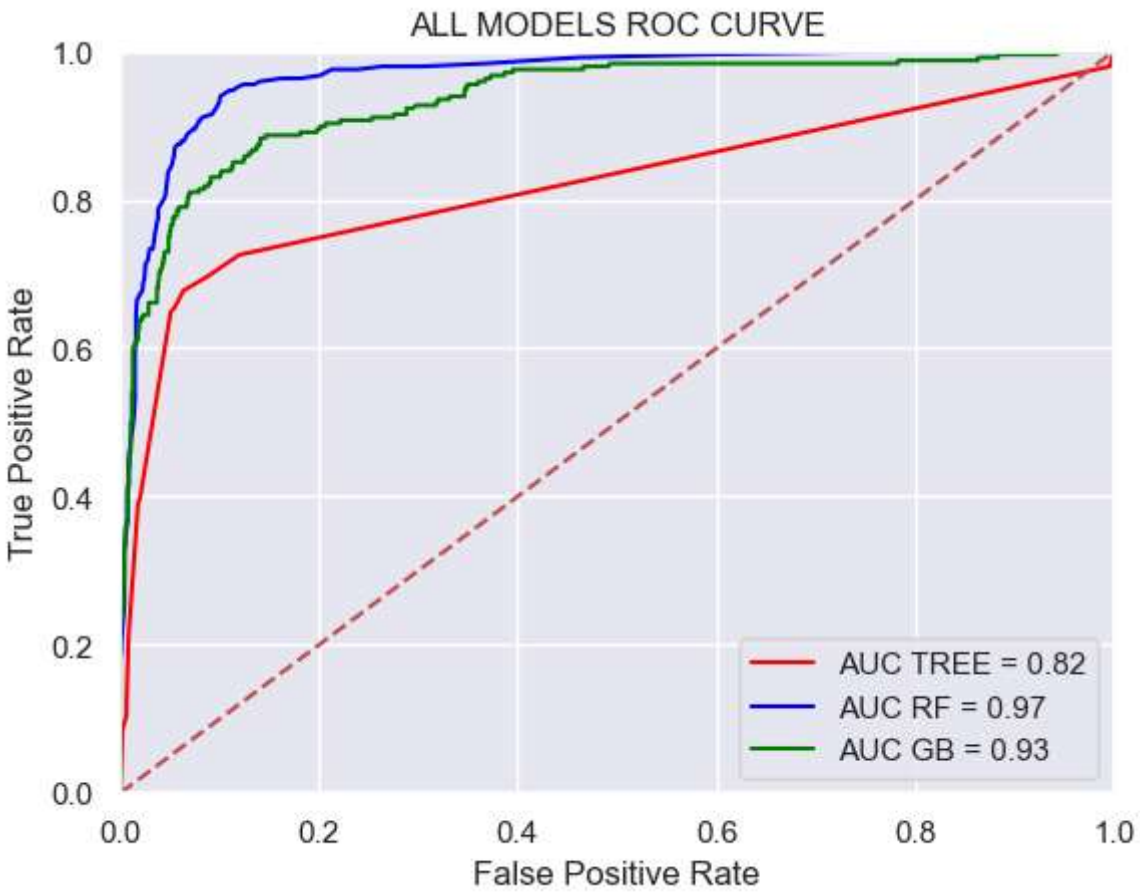
```
plt.title("GB ROC CURVE")
plt.plot(fpr_train_GB, tpr_train_GB, "b", label = "AUC TRAIN = %0.2f" % roc_auc_tra
plt.plot(fpr_test_GB, tpr_test_GB, label = "AUC TEST = %0.2f" % roc_auc_test_GB, co
plt.legend(loc = "lower right")
plt.plot([0,1], [0,1], "r--")
plt.xlim([0,1])
plt.ylim([0,1])
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.show()
```



ALL Models

In [202…

```
plt.title("ALL MODELS ROC CURVE")
plt.plot(fpr_tree, tpr_tree, label = "AUC TREE = %0.2f" % auc_tree, color = "Red")
plt.plot(fpr_RF, tpr_RF, label = "AUC RF = %0.2f" % auc_RF, color = "Blue")
plt.plot(fpr_GB, tpr_GB, label = "AUC GB = %0.2f" % auc_GB, color = "Green")
plt.legend(loc = "lower right")
plt.plot([0,1], [0,1], "r--")
plt.xlim([0,1])
plt.ylim([0,1])
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.show()
```



In [205…

```
print("Root Mean Square Average for Loan Amounts")
print("TREE", RMSE_tree)
print("RF", RMSE_RF)
print("GB", RMSE_GB)
```

```
Root Mean Square Average for Loan Amounts
TREE 5410.350018411686
RF 3225.6898950862105
GB 2614.670782701094
```